

ANTDESIGNER REDESIGN - SINGLE FILE FOR AI ARENA

HOW TO USE (for the human): give Arena (1) your real repo (GitHub `kara-nawzad/Antenna-AI-Project` , or a ZIP with `10_web_app.py` , `engine_v2.py` , `antenna_viz.py` ; run `git lfs pull` first or the scaler .pkl files are only pointers) and (2) THIS file. Tell it: "Read this file completely and follow it exactly."

INSTRUCTIONS FOR ARENA: This file has two parts: PART A = the task and safety rules, PART B = two files you must create verbatim in the repo. Create the files from PART B exactly as written (do not "improve" them before Step 0 and Step 2 checks), then follow PART A.

=====:

PART A - TASK

ROLE

You are a careful senior front-end engineer working on an existing, working Streamlit application: "**AI-Based CPW Microstrip Antenna Synthesis**" (repo `kara-nawzad/Antenna-AI-Project` , main file `10_web_app.py` , engine in `engine_v2.py` , drawing in `antenna_viz.py`). Your job is a **visual redesign only**. The engineering must come out byte-for-byte equivalent in behaviour.

GOAL

Apply the **AntDesigner** design system (dark, cinematic, electric-purple) to the app. I am giving you:

- `antdesigner_theme.py` - theme module (CSS, animated energy-field background, navbar, hero with isometric antenna SVG, matplotlib dark styling helper)
- `.streamlit/config.toml` - native dark/purple theme

Brand (strict): charcoal #1F2937, electric purple #6366F1, background #080B14, surface #111624, text #F8F AFC, secondary text #A1A1AA, borders `rgba(255,255,255,.10)`. Fonts: Sora (headings, light/regular weights) + Inter (body). **No green, orange or cyan as accents** (the current app uses orange #ff9f43 and green fills: replace them). Wordmark: "Ant" white/bold + "Designer" purple/regular, as in the approved logo. Keep the purple vivid but controlled; keep text readable.

ABSOLUTE RULES (violating any = stop and report)

1. **Do not change behaviour.** Forbidden to edit logic in: `engine_v2.py` ; every `forward_*` function; `run_inverse_solver` ; `run_engine_self_test` ; `run_startup_healthcheck` ; `load_brains` ; `load_data` ; `_download_hf` ; validation functions; `clamp_5_40` /sync callbacks; AETHER/Groq code; `Dockerfile/fly.toml/requirements` (unless I ask).
2. **Do not rename or remove** any `st.session_state` key (`fwd_res` , `inv_res` , `self_test_stats` , `inv_test_summary` , `inv_test_family` , `messages` , `lp_val` , ...) or any widget key= (`lp_slider` , `lp_input` , `wp` equivalents, etc.). Do not change widget ranges, defaults, labels' meaning, or step values.
3. **Do not change geometry** in `antenna_viz.py` (constants `lf` , `wf` , `g` , `ws` , `ls` , `g_ymax` and all 12 shape branches). Colours/fonts/figure facecolor only.
4. **Never fabricate data.** No hard-coded engineering numbers in the UI. The hero antenna is *illustrative* and must keep its "Illustrative geometry - not computed output" label. Do not add claims about accuracy/capabilities that the project description does not support.
5. **Work on a new git branch** `design/antdesigner-ui` created from the current `main` . Do not push to main, do not merge the unmerged `arena/01a04465-...` branch, do not deploy (no `fly deploy`), do not touch the Hugging Face repo, do not print or commit secrets (GROQ_API_KEY).
6. **Do nothing without checking.** Before each edit, read the exact current code you are about to change. After each edit, run the check listed for that step. If a check fails, fix it before continuing. If something is ambiguous or differs from this prompt, **stop and ask me instead of guessing**. Anything you could not verify must be labelled `NOT VERIFIED` in your report, never assumed.

STEP 0 - RECON (read-only; no edits)

Read every file in the repo. Then output a short report answering, with line numbers:

- a) Streamlit version in use (requirements has no pins - check installed/pip freeze). Does `st.markdown(unsafe_allow_html=True)` pass inline `<svg>` and `<style>` in that version?
 - b) Where the big CSS+navbar `st.markdown` block, the hero, the Introduction panel, the "System Overview" cards, the six section anchors, the footer and the `components.html` JS block are. Confirm the section ids really are `overview` , `introduction` , `intel` , `simulator` , `designer` , `assistant` , and list the real CSS class names (my theme assumes: `navbar` , `nav-content` , `nav-brand` , `nav-links` , `nav-item` , `nav-home-link` , `section-anchor` , `intro-panel` , `info-card` , `result-banner` , `intro-credits` , `section-title` , `section-subtitle` , `status-heading` , `footer-text`). **Any mismatch: tell me and adapt the theme, do not silently ignore it.**
 - c) Every place with hard-coded colours (CSS, matplotlib calls, `antenna_viz.py`) - list them.
 - d) Which Streamlit element receives the page scroll (`[data-testid="stMain"]` vs window) in the installed version.
- STOP after the report and wait for my "go" if anything differs from the above.

STEP 1 - BASELINE (before touching anything)

Run the app locally (`streamlit run 10_web_app.py`). Record to `baseline.json` the exact outputs for:

- Forward: $L_p=22.5$, $W_p=18.0$, default shape, default ML-refine setting -> resonance, bandwidth, S11 min, VSWR, gain, efficiency, engine mode string, nearest-anchor line.
 - Inverse: 3.5 GHz, Balanced Profile, Rectangular -> optimal L_p , W_p , accuracy, gain, BW.
- Take full-page screenshots (desktop 1440px, mobile 390px) as `before_*.png` . If the models cannot be downloaded in your sandbox, say so and record what mode the app falls back to (CSV-only) - do not fake values.

STEP 2 - ADD FILES

Add `antdesigner_theme.py` next to `10_web_app.py` . Merge `.streamlit/config.toml` (if one already exists, merge the `[theme]` keys, do not overwrite other settings). Check: `python -c "import antdesigner_theme"` works; the Dockerfile `COPY . .` will include the file (verify, don't assume).

STEP 3 - WIRE IT IN (small, reviewable edits)

In `10_web_app.py` :

1. Immediately after `st.set_page_config(...)` call `antdesigner_theme.apply_theme()` .
 2. Remove the old ~370-line CSS block, the old navbar HTML, and the old `components.html` nav/scroll-spy JS (it targets `window.top` ; the new script handles smooth-scroll + scroll-spy and targets Streamlit's real scroll container). Keep the logo base64 logic; pass it to `render_navbar(logo_data_uri)` .
 3. Replace the old hero with `st.markdown(render_hero(), unsafe_allow_html=True)` . The new hero already contains the `overview` anchor - make sure the id is not duplicated. Keep the Google Drive link and the Introduction panel text unchanged (credits, supervisor, limitations paragraph stay word-for-word).
 4. Keep sections Introduction, System Overview, Forward Simulation, Engine Validation, Inverse Design, Inverse Validation, AETHER, Footer **in the same order with the same anchor ids**; only restyle through CSS.
 5. Matplotlib (S11 + VSWR): the theme file contains `plot_s11_vswr(freqs_ghz, s11_db, vswr, fr_ghz, s11_min_db)` which draws two side-by-side panels: S11 vs GHz (purple line, dashed -10 dB line, shaded band where $S_{11} \leq -10$ dB, white resonance dot with label) and VSWR vs GHz (same shaded band, dotted $VSWR=2$ line, y-axis 1..5). Replace the existing two-panel figure with it ONLY IF you can pass the exact same arrays the current code plots (verify names, units, shapes). Otherwise keep the existing plotting code and only recolour it via `style_figure(fig)` and `PLOT` . The plotted data must be the engine's real arrays - never regenerate or smooth them. Stack the panels vertically on narrow screens if feasible; threshold lines must be distinguishable by dash style + legend text, not colour alone. Remember -10 dB ~ VSWR 1.92 ($VSWR=2 \sim -9.54$ dB): keep both reference lines but label them honestly. The dataset has no phase data, so do NOT add Smith charts or impedance plots.
 6. `antenna_viz.py` : recolour only (dark figure facecolor #11624, substrate #1F2937, metal in light purple/white tints, dimension arrows white/purple, no orange). Diff must show colour literals only.
 7. Status heading: do not rely on colour alone - keep the text "WORKING / NOT WORKING" (+ an icon). A muted red for the NOT WORKING/error state is allowed as a functional state colour; no green.
- Check after each sub-step: `python -m py_compile 10_web_app.py antdesigner_theme.py antenna_viz.py` ; app boots with no Python traceback and no browser console errors.

STEP 4 - VERIFY (all must pass; show evidence for each)

1. **Regression**: repeat the Step 1 inputs; every number and engine-mode string must be identical to `baseline.json` .
2. **Diff audit**: `git diff --stat` and `git diff` - confirm no logic changes, `engine_v2.py` untouched, no session-state/widget-key changes. Paste the list of changed files and a one-line justification for each.
3. **Browser test** (Playwright/headless Chrome if available, otherwise mark NOT VERIFIED): desktop 1440 and mobile 390 screenshots; no horizontal scrolling; nav turns translucent on scroll; each nav link scrolls to the right section and the active underline follows; hero CTAs jump to Inverse (`#designer`) and Forward (`#simulator`); the animated background is visible behind the hero and faint behind app sections; text contrast is readable everywhere.
4. **Click every control**: Initiate Simulation, ML-refine checkbox, sliders and number boxes stay in sync, shape select, Run Self-Test, Synthesize Optimal Geometry, Inverse Self-Test (+ CSV download), AETHER chat (with key and without key -> info box).
5. **Accessibility/perf**: `prefers-reduced-motion` stops animation (static field frame); visible focus rings; canvas pauses when the tab is hidden; page remains smooth on a throttled CPU (report the FPS or mark NOT VERIFIED); no extra network requests besides Google Fonts.
6. **Docker**: `docker build` succeeds (if Docker is unavailable, say NOT VERIFIED). Do not deploy.

STEP 5 - REPORT (required format)

1. Branch name and commit list. 2. Before/after screenshots. 3. A table of every Step 4 check: PASS / FAIL / NOT VERIFIED with evidence. 4. Anything that differed from this prompt and how you handled it. 5. Known problems you noticed but did NOT fix (e.g. UI range 5-40 mm vs trained 10-35 mm; band-edge "resonance" values; dip specialist missing; wrong HF_REPO inside `engine_v2.py`) - list them, do not change them unless I ask. 6. Exact rollback

```
command ( git checkout main ).
```

Finish only when every check is PASS, or clearly explain each FAIL / NOT VERIFIED and wait for me.

PART B - FILES TO CREATE VERBATIM

B1. .streamlit/config.toml (merge with an existing one, never overwrite other settings)

```
# Native Streamlit theme: makes sliders, selects, number inputs, chat etc. adopt the brand
# automatically. MERGE with an existing config.toml if there is one (do not overwrite blindly).
[theme]
base = "dark"
primaryColor = "#6366F1"
backgroundColor = "#080B14"
secondaryBackgroundColor = "#111624"
textColor = "#F8FAFC"
font = "sans serif"
```

B2. antdesigner_theme.py (place next to 10_web_app.py)

```
"""
antdesigner_theme.py - AntDesigner visual layer for the Streamlit antenna app.

PRESENTATION ONLY. This module does not import, call or modify the engine, the models,
the CSV database, validation logic, Groq/AETHER logic or session-state keys.

Public API (call order in 10_web_app.py, right after st.set_page_config):
    apply_theme()          # CSS + animated background + nav behaviour
    st.markdown(render_navbar(), unsafe_allow_html=True)
    st.markdown(render_hero(), unsafe_allow_html=True)
    ...existing sections, unchanged...
    style_figure(fig)      # optional: dark-mode matplotlib figures
Colour constants for matplotlib are in PLOT.
"""
import json
import streamlit as st
import streamlit.components.v1 as components

# ----- brand
PALETTE = dict(bg="#080B14", surface="#111624", charcoal="#1F2937", purple="#6366F1",
               purple_soft="#818CF8", text="#F8FAFC", text2="#A1A1AA")

PLOT = dict(bg="#111624", axes="#111624", grid=(1, 1, 1, 0.08), text="#A1A1AA", title="#F8FAFC",
            s1l="#818CF8", vswr="#C7C9F5", threshold=(1, 1, 1, 0.45), fill=(0.39, 0.40, 0.94, 0.20),
            marker_face="#F8FAFC", marker_edge="#6366F1")

# Existing section ids (kept identical to the current app so nothing else breaks)
NAV = [("Home", "overview"), ("Introduction", "introduction"), ("Method", "intel"),
       ("Forward", "simulator"), ("Inverse", "designer"), ("AETHER", "assistant")]
DRIVE_URL = "https://drive.google.com/drive/folders/1DtTck0vSXXKJWLYIGdFh75lOmcNlXlypJ"

def _flat(html: str) -> str:
    """st.markdown treats indented/blank lines as markdown blocks -> flatten the HTML."""
    return "".join(line.strip() for line in html.splitlines())

# ----- CSS
THEME_CSS = """
@import url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;600;700&family=Inter:wght@400;500;600&display=swap');
:root{--bg:#080B14;--surf:#111624;--char:#1F2937;--p:#6366F1;--p2:#818CF8;--tx:#F8FAFC;--tx2:#A1A1AA;
--bd:rgba(255,255,255,.10);--ease:cubic-bezier(.22,.8,.3,1)}
html,body{background:var(--bg)!important;color:var(--tx);font-family:Inter,system-ui,sans-serif}
.stApp,[data-testid="stAppViewContainer"],[data-testid="stMain"],section.main{background:transparent!important}
[data-testid="stAppViewContainer"]{position:relative;z-index:1}
[data-testid="stHeader"],[data-testid="stToolbar"],#MainMenu,footer{display:none!important}
.block-container{max-width:1200px;padding-top:0!important;padding-bottom:4rem}
h1,h2,h3,h4{font-family:Sora,sans-serif!important;font-weight:400!important;letter-spacing:-.02em;color:var(--tx)!important}
p,li,label,span,div{font-family:Inter,system-ui,sans-serif}
:focus-visible{outline:2px solid var(--p)!important;outline-offset:3px}
```

```

/* ----- navbar (keeps legacy class names: navbar, nav-item, nav-home-link) ----- */
.navbar{position:fixed;top:0;left:0;right:0;z-index:1000;border-bottom:1px solid transparent;
transition:background .3s;border-color .3s,backdrop-filter .3s}
.navbar.scrolled(background:rgba(8,11,20,.74);backdrop-filter:blur(14px);-webkit-backdrop-filter:blur(14px);border-color:var(--bd))
.nav-content{max-width:1240px;margin:0 auto;padding:14px 24px;display:flex;align-items:center;gap:28px}
.nav-brand{display:flex;align-items:center;gap:12px;color:var(--tx);text-decoration:none}
.nav-brand.wm{font-family:Sora,sans-serif;font-size:20px;letter-spacing:-.01em}
.nav-brand.wm b{font-weight:700;color:#fff}.nav-brand.wm span{font-weight:400;color:var(--p)}
.nav-brand.small{display:block;font-size:10px;letter-spacing:.12em;color:var(--tx2);text-transform:uppercase;margin-top:1px}
.nav-links{display:flex;gap:6px;margin-left:auto}
.nav-item(position:relative;padding:8px 14px;border-radius:999px;font-size:14px;color:var(--tx2)!important;text-decoration:none!important
transition:color .25s;background .25s)
.nav-item:hover{color:var(--tx)!important;background:rgba(255,255,255,.04)}
.nav-item.active{color:var(--tx)!important}
.nav-item.active::after{content:"";position:absolute;left:14px;right:14px;bottom:2px;height:2px;background:var(--p);border-radius:2px}
.section-anchor{display:block;position:relative;top:-90px;visibility:hidden;height:0}
@media(max-width:900px){.nav-links{display:none}.nav-brand.small{display:none}}

/* ----- hero ----- */
.ad-hero{min-height:100vh;display:grid;grid-template-columns:1fr 1.1fr;gap:24px;align-items:center;padding:110px 0 56px}
.ad-eye{font-size:12px;letter-spacing:.22em;color:var(--p2);font-weight:500;margin:0 0 20px}
.ad-h1{font-family:Sora,sans-serif;font-weight:300;font-size:clamp(38px,5.2vw,72px);line-height:1.06;letter-spacing:-.025em;margin:0 0 22px}
.ad-h1 em{font-style:normal;color:var(--p);font-weight:400}
.ad-sub{font-size:clamp(15px,1.25vw,18px);line-height:1.65;color:var(--tx2);max-width:540px;margin:0 0 30px}
.ad-ctas{display:flex;gap:14px;flex-wrap:wrap;margin-bottom:26px}
.ad-btn{display:inline-flex;align-items:center;gap:10px;padding:12px 22px;min-height:44px;border-radius:999px;font-weight:500;font-size:14px;border:1px solid var(--bd);color:var(--tx)!important;text-decoration:none!important;transition:transform .25s var(--ease),box-shadow .25s var(--p)}
.ad-btn.pri{background:var(--p);border-color:var(--p)}
.ad-btn.pri:hover{transform:translateY(-2px);box-shadow:0 8px 28px -6px rgba(99,102,241,.7);filter:brightness(1.08)}
.ad-btn.sec{background:rgba(255,255,255,.03)}
.ad-btn.sec:hover{transform:translateY(-2px);border-color:rgba(99,102,241,.6);background:rgba(99,102,241,.08)}
.ad-chips{display:flex;gap:10px;flex-wrap:wrap;margin:0 0 18px}
.ad-chip{font-size:12px;color:var(--tx2);border:1px solid var(--bd);border-radius:999px;padding:6px 12px;background:rgba(17,22,36,.6)}
.ad-link{font-size:13px;color:var(--tx2)!important;text-decoration:none!important;border-bottom:1px solid rgba(99,102,241,.5)}
.ad-link:hover{color:var(--tx)!important}
.ad-note{font-size:11px;color:var(--tx2);text-align:center;margin-top:4px;opacity:.8}
.ad-viz svg{width:100%;height:auto;display:block;overflow:visible;animation:ad-float 9s ease-in-out infinite}
@media(max-width:900px){.ad-hero{grid-template-columns:1fr;padding-top:96px;min-height:auto}}
.ad-rv(opacity:0;transform:translateY(18px);animation:ad-up .9s var(--ease) forwards)
@keyframes ad-up{to{opacity:1;transform:none}}
@keyframes ad-float{50%{transform:translateY(-8px)}}
.ad-fade{opacity:0;animation:ad-fi .9s ease forwards}
@keyframes ad-fi{to{opacity:1}}
.ad-dr{stroke-dasharray:1;stroke-dashoffset:1;animation:ad-dr 1.4s ease forwards}
@keyframes ad-dr{to{stroke-dashoffset:0}}
.ad-flow{stroke-dasharray:.08 .92;animation:ad-fl 2.8s linear infinite}
@keyframes ad-fl{from{stroke-dashoffset:1}to{stroke-dashoffset:0}}
.ad-lab{font:500 10.5px Inter,sans-serif;fill:#C7C9F5;letter-spacing:.04em}

/* ----- content panels (legacy class names restyled) ----- */
.intro-panel,.info-card,.result-banner,.intro-credits(background:rgba(17,22,36,.84)!important;border:1px solid var(--bd)!important;
border-radius:20px!important;backdrop-filter:blur(8px);color:var(--tx)!important;box-shadow:none!important)
.intro-panel{padding:44px!important;max-width:960px;margin:0 auto}
.intro-panel h1,.intro-panel h2,.intro-panel h3,.section-title{color:var(--tx)!important;font-family:Sora,sans-serif!important;font-weight:300}
.section-title{font-size:clamp(26px,3vw,40px);text-align:center;margin:72px 0 8px}
.section-subtitle{color:var(--tx2)!important;text-align:center;margin-bottom:28px}
.info-card{padding:24px!important;transition:transform .25s var(--ease),border-color .25s}
.info-card:hover{transform:translateY(-3px);border-color:rgba(99,102,241,.5)!important}
.status-heading{font-family:Sora,sans-serif;font-weight:400;text-align:center;color:var(--tx)}
.footer-text{color:var(--tx2)!important;text-align:center;font-size:13px;border-top:1px solid var(--bd);padding-top:28px;margin-top:64px}

/* ----- Streamlit widgets ----- */
.stButton>button,.stDownloadButton>button{background:var(--p)!important;color:#fff!important;border:1px solid var(--p)!important;border-radius:16px;
padding:.6rem 1.5rem!important;min-height:44px;font-weight:500!important;transition:transform .25s var(--ease),box-shadow .25s,filter .25s}
.stButton>button:hover,.stDownloadButton>button:hover{transform:translateY(-2px);box-shadow:0 8px 28px -6px rgba(99,102,241,.7);filter:brightness(1.08)}
.stButton>button:active{transform:translateY(0)}
[data-baseweb="input"],[data-baseweb="base-input"],[data-baseweb="select"]>div{background:var(--surf)!important;border-color:var(--bd)!important}
[data-baseweb="input"]:focus-within,[data-baseweb="select"]>div:focus-within{border-color:var(--p)!important;box-shadow:0 0 1px var(--p)}
input,textarea{color:var(--tx)!important}
[data-testid="stMetric"]{background:rgba(17,22,36,.88);border:1px solid var(--bd);border-radius:16px;padding:16px 18px;transition:border-color .25s}
[data-testid="stMetric"]>div{border-color:rgba(99,102,241,.5);transform:translateY(-2px)}
[data-testid="stMetricLabel"]{color:var(--tx2)!important;text-transform:uppercase;letter-spacing:.08em;font-size:11px!important}
[data-testid="stMetricValue"]{color:var(--tx)!important;font-family:Sora,sans-serif!important;font-weight:400!important}
[data-testid="stAlert"]{background:rgba(99,102,241,.08)!important;border:1px solid rgba(99,102,241,.35)!important;border-radius:14px!important}
[data-testid="stChatMessage"]{background:var(--surf)!important;border:1px solid var(--bd);border-radius:16px}

```

```

[data-testid="stChatInput"]{background:var(--surf)!important;border-radius:16px!important}
[data-testid="stDataFrame"],[data-testid="stTable"]{border:1px solid var(--bd);border-radius:14px;overflow:hidden}
[data-testid="stImage"] img,[data-testid="stPyplot"] img{border-radius:16px;border:1px solid var(--bd)}
hr{border-color:var(--bd)!important}
@media(prefers-reduced-motion:reduce){*{animation-duration:.01ms!important;animation-delay:0s!important;animation-iteration-count:1!important}}

# ----- JS (runs in the PARENT page)
# Streamlit scrolls inside [data-testid="stMain"], NOT window, so scroll code targets that element.
PARENT_JS = r"""
(function(){
  const d=document, W=window;
  const IDS=["overview","introduction","intel","simulator","designer","assistant"];
  const rm=W.matchMedia('(prefers-reduced-motion: reduce)').matches;
  const scroller=()=>d.querySelector('[data-testid="stMain"]')||d.querySelector('section.main');
  let sc=0,mx=0,my=0,tx=0,ty=0;

  /* ---- nav: scrolled state, scroll-spy, smooth anchors ---- */
  function onScroll(){
    const m=scroller(); sc=m?m.scrollTop:W.scrollY;
    const nav=d.querySelector('.navbar'); if(nav) nav.classList.toggle('scrolled',sc>40);
    let cur=IDS[0];
    for(const id of IDS){const e=d.getElementById(id); if(e && e.getBoundingClientRect().top<=140) cur=id;}
    d.querySelectorAll('.nav-item').forEach(a=>a.classList.toggle('active',a.getAttribute('data-target')==cur));
  }
  d.addEventListener('click',function(ev){
    const a=ev.target.closest(&ev.target.closest('.nav-item,.ad-go'); if(!a) return;
    const id=a.getAttribute('data-target'); const el=id&&d.getElementById(id); if(!el) return;
    ev.preventDefault(); el.scrollIntoView({behavior:rm?'auto':'smooth',block:'start'});
  },true);
  let tries=0; const iv=setInterval(function(){
    const m=scroller(); if(m){m.addEventListener('scroll',onScroll,{passive:true}); onScroll(); clearInterval(iv);}
    else if(++tries>60){clearInterval(iv);}
  },300);

  /* ---- energy field canvas ---- */
  let cv=d.getElementById('ad-field');
  if(!cv){cv=d.createElement('canvas'); cv.id='ad-field'; d.body.appendChild(cv);}
  cv.setAttribute('aria-hidden','true');
  cv.style.cssText='position:fixed;inset:0;width:100%;height:100%;z-index:0:pointer-events:none';
  const x=cv.getContext('2d'); let Wd,Hd,D,lines=[],streaks=[],vis=true,t0=performance.now();
  function build(){
    D=Math.min(W.devicePixelRatio||1,2); Wd=W.innerWidth; Hd=W.innerHeight; cv.width=Wd*D; cv.height=Hd*D;
    const n=Wd<700?24:Wd<1100?38:56; lines=[]; streaks=[];
    for(let i=0;i<n;i++){
      const u=i/(n-1), side=u<.5?0:1, k=Math.abs(u-.5)*2;
      lines.push({sx:Wd*(side?1.05:-.05), sy:Hd*(.05+(1-k)*.9+Math.random()*1), ex:Wd*(.5+(Math.random()-).5)*.08, ey:Hd*.8,
        cx:Wd*(.5+(side?.25:-.25)*(1+Math.random()*1.6)), cy:Hd*(.1+Math.random()*1.9), a:.08+Math.random()*1.2,
        ph:Math.random()*6.28, sp:.15+Math.random()*1.25, z:.3+Math.random()*1.7});
      if(i%2===0) streaks.push({l:i,t:Math.random(),v:.04+Math.random()*1.08,a:.4+Math.random()*1.5});
    }
  }
  const bz=(l,t,w)=>{const cx=l.cx+w,cy=l.cy+w*.6,u=1-t; return [u*1.sx+2*u*t*cx+t*1.ex, u*1.sy+2*u*t*cy+t*1.ey];};
  function frame(now){
    if(!vis){requestAnimationFrame(frame);return;}
    const t=(now-t0)/1000; mx+=(tx-mx)*.04; my+=(ty-my)*.04;
    x.setTransform(D,0,0,D,0,0); x.clearRect(0,0,Wd,Hd);
    const fade=Math.max(.22,1-sc/(Hd*.9)); /* stays faint behind the app sections */
    const fx=Wd*.5+mx*18, fy=Hd*.8-Math.min(sc,Hd)*.1+my*10;
    x.globalCompositeOperation='lighter';
    for(const l of lines){
      const w=Math.sin(t*1.sp+1.ph)*28*1.z+mx*14*1.z; x.beginPath();
      for(let k=0;k<=24;k++){const p=bz(l,k/24,w); k?x.lineTo(p[0],p[1]):x.moveTo(p[0],p[1]);}
      const g=x.createLinearGradient(1.sx,1.sy,1.ex,1.ey);
      g.addColorStop(0,'rgba(99,102,241,0)'); g.addColorStop(.6,'rgba(129,131,245,'+(1.1*fade)+')'); g.addColorStop(1,'rgba(165,160,255,'+(1.1*fade)+')');
      x.strokeStyle=g; x.lineWidth=.6*1.z*.7; x.stroke();
    }
    if(!rm) for(const s of streaks){
      s.t+=s.v*.016; if(s.t>1) s.t=0; const l=lines[s.l], w=Math.sin(t*1.sp+1.ph)*28*1.z+mx*14*1.z; x.beginPath();
      for(let k=0;k<=6;k++){const tt=Math.max(0,s.t-1+k*.0167),p=bz(l,tt,w); k?x.lineTo(p[0],p[1]):x.moveTo(p[0],p[1]);}
      x.strokeStyle='rgba(196,198,255,'+(1.1*s*Math.sin(s.t*Math.PI)*fade)+')'; x.lineWidth=1.4; x.stroke();
    }
    const r=Math.min(Wd,Hd)*(.42+.03*Math.sin(t*.6))*(.85+.15*Math.sin(t*1.2));
    let gr=x.createRadialGradient(fx,fy,0,fx,fy,r);
    gr.addColorStop(0,'rgba(165,160,255,'+(.55*fade)+')'); gr.addColorStop(.12,'rgba(99,102,241,'+(.32*fade)+')');
  }
}

```

```

gr.addColorStop(.5,'rgba(99,102,241,+.07*fade)+'); gr.addColorStop(1,'rgba(99,102,241,0)');
x.fillStyle=gr; x.fillRect(0,0,Wd,Hd); x.globalCompositeOperation='source-over';
if(!rm) requestAnimationFrame(frame);
}
build(); W.addEventListener('resize',function(){build(); if(rm) frame(performance.now());});
if(W.matchMedia('(pointer:fine)').matches && !rm) d.addEventListener('pointermove',function(e){tx=e.clientX/W.innerWidth-.5; ty=e.clientY/H.d.addEvent
d.addEventListener('visibilitychange',function(){vis=!d.hidden;});
requestAnimationFrame(frame);
})();
"""

# ----- antenna illustration (Python-generated SVG)
def antenna_svg() -> str:
    """Isometric microstrip-patch illustration. Decorative/illustrative geometry only - NOT engine output."""
    S, CX, CY, c, s = 2.15, 300, 262, 0.866, 0.5

    def P(x, y, z):
        return (CX + (x - y) * c * S, CY + (x + y) * s * S - z * S * 1.6)

    def pts(a):
        return " ".join(f"{p[0]:.1f},{p[1]:.1f}" for p in a)

    def poly(a, fill, stroke, w, extra=""):
        return f'<polygon points="{pts(a)}" fill="{fill}" stroke="{stroke}" stroke-width="{w}" stroke-linejoin="round" {extra}/>'

    def box(x0, y0, x1, y1, z0, z1, top, left, right, stroke, w, delay):
        T = [P(x0, y0, z1), P(x1, y0, z1), P(x1, y1, z1), P(x0, y1, z1)]
        R = [P(x1, y0, z1), P(x1, y1, z1), P(x1, y1, z0), P(x1, y0, z0)]
        L = [P(x0, y1, z1), P(x1, y1, z1), P(x1, y1, z0), P(x0, y1, z0)]
        return (f'<g class="ad-fade" style="animation-delay:{delay}s">'
                + poly(L, left, stroke, w) + poly(R, right, stroke, w) + poly(T, top, stroke, w) + "</g>")

    h = ""
    g = ""
    for i in range(-90, 91, 15):
        g += "M%.1f %.1fL%.1f %.1f" % (*P(i, -70, 0), *P(i, 70, 0))
    for j in range(-70, 71, 14):
        g += "M%.1f %.1fL%.1f %.1f" % (*P(-90, j, 0), *P(90, j, 0))
    h += f'<path d="{g}" stroke="rgba(255,255,255,.07)" fill="none" class="ad-fade" style="animation-delay:.3s"/>'
    h += box(-60, -46, 60, 46, -3, 0, "#1b2233", "#10151f", "#161c2a", "rgba(255,255,255,.22)", 1, .5)
    h += box(-60, -46, 60, 46, 0, 10, "rgba(99,102,241,.14)", "rgba(31,41,55,.95)", "rgba(40,52,72,.95)", 1, .9)
    patch = [P(-32, -24, 10.6), P(32, -24, 10.6), P(32, 24, 10.6), P(-32, 24, 10.6)]
    feed = [P(-60, -3, 10.6), P(-32, -3, 10.6), P(-32, 3, 10.6), P(-60, 3, 10.6)]
    h += (f'<g class="ad-fade" style="animation-delay:1.4s">'
          + poly(patch, "rgba(99,102,241,.34)", "#8B8DF7", 1.6)
          + poly(feed, "rgba(99,102,241,.5)", "#8B8DF7", 1.4) + "</g>")
    a, b = P(-60, 0, 10.7), P(-32, 0, 10.7)
    h += (f'<path d="M{a[0]:.1f} {a[1]:.1f}L{b[0]:.1f} {b[1]:.1f}" stroke="#C7C9F5" stroke-width="1.6" fill="none" '
          f'pathLength="1" class="ad-flow" style="animation-delay:2.2s;filter:drop-shadow(0 0 6px #6366F1)"/>')

    def dim(p, q, text, lx, ly, delay):
        return (f'<g class="ad-fade" style="animation-delay:{delay}s"><path d="M{p[0]:.1f} {p[1]:.1f}L{q[0]:.1f} {q[1]:.1f}" '
                f'stroke="#8B8DF7" stroke-width="1" fill="none" stroke-dasharray="3 3"/>'
                f'<circle cx="{p[0]:.1f}" cy="{p[1]:.1f}" r="2.2" fill="#8B8DF7"/><circle cx="{q[0]:.1f}" cy="{q[1]:.1f}" r="2.2" fill="#8B8DF7"/>'
                f'<text class="ad-lab" x="{lx:.1f}" y="{ly:.1f}" text-anchor="middle">{text}</text></g>')

    m = P(0, 38, 10.6); h += dim(P(-32, 32, 10.6), P(32, 32, 10.6), "Lp", m[0] - 14, m[1] + 14, 2.0)
    m = P(46, 0, 10.6); h += dim(P(40, -24, 10.6), P(40, 24, 10.6), "Wp", m[0] + 16, m[1] + 4, 2.3)

    def callout(x, y, tx, ty, text, delay, anchor):
        off = -6 if anchor == "end" else 6
        return (f'<g class="ad-fade" style="animation-delay:{delay}s"><path d="M{x:.1f} {y:.1f}L{tx:.1f} {ty:.1f}" '
                f'stroke="rgba(255,255,255,.35)" fill="none"/><text class="ad-lab" x="{tx + off:.1f}" y="{ty + 3:.1f}" text-anchor="{anch">{text}</text></g>')

    p1, p2, p3, p4 = P(10, -24, 10.6), P(-58, 0, 10.6), P(60, -20, 5), P(60, 30, -1.5)
    h += callout(p1[0], p1[1], p1[0] + 30, p1[1] - 62, "Radiating patch", 3.0, "start")
    h += callout(p2[0], p2[1], p2[0] - 30, p2[1] - 34, "Feed line", 3.2, "end")
    h += callout(p3[0], p3[1], p3[0] + 54, p3[1] - 14, "Dielectric substrate", 3.4, "start")
    h += callout(p4[0], p4[1], p4[0] + 40, p4[1] + 34, "Ground plane", 3.6, "start")
    return ('<svg viewBox="0 0 600 440" role="img" aria-label="Isometric illustration of a microstrip patch antenna: '
            f'patch, feed line, substrate and ground plane">{h}</svg>')

# ----- HTML blocks

```

```

def render_navbar(logo_data_uri: str = "") -> str:
    """Fixed nav. Pass the existing base64 logo data-URI if you want to keep the SPU logo."""
    logo = (f'' if logo_data_uri else "")
    links = "".join(f'<a class="nav-item" href="#{i}" data-target="{i}"><n></a>' for n, i in NAV)
    return _flat(f"""
<div class="navbar"><div class="nav-content">
    <a class="nav-brand nav-home-link" href="#overview" data-target="overview">{logo}
        <div><div class="wm"><b>Ant</b><span>Designer</span></div><small>Sulaymaniyah Polytechnic University</small></div></a>
        <div class="nav-links">{links}</div>
</div></div>""")

def render_hero() -> str:
    arrow = ('<svg width="16" height="16" viewBox="0 0 16 16" fill="none" stroke="currentColor" stroke-width="1.8">'
        '<path d="M3 8h10M9 4l4 4 4 4"/></svg>')
    return _flat(f"""
<div id="overview" class="section-anchor"></div>
<section class="ad-hero">
    <div>
        <p class="ad-eye ad-rv" style="animation-delay:.1s">AI-BASED CPW ANTENNA SYNTHESIS</p>
        <h1 class="ad-h1 ad-rv" style="animation-delay:.25s">Design Smarter Antennas. <em>In Seconds.</em></h1>
        <p class="ad-sub ad-rv" style="animation-delay:.45s">Predict the performance of a CPW-fed microstrip patch antenna from its dimen-
or find the dimensions that reach your target frequency between 1 and 7 GHz &mdash; anchored to a database of CST simulations.</p>
        <div class="ad-ctas ad-rv" style="animation-delay:.6s">
            <a class="ad-btn pri ad-go" href="#designer" data-target="designer">Synthesize a Design {arrow}</a>
            <a class="ad-btn sec ad-go" href="#simulator" data-target="simulator">Predict Performance</a>
        </div>
        <div class="ad-chips ad-rv" style="animation-delay:.7s"><span class="ad-chip">12 patch shapes</span><span class="ad-chip">4 shape
            <span class="ad-chip">S11 &amp; VSWR curves</span><span class="ad-chip">CST-anchored engine</span></div>
        <a class="ad-link ad-rv" style="animation-delay:.8s" href="{DRIVE_URL}" target="_blank" rel="noopener noreferrer">Research papers
        </div>
        <div class="ad-viz">{antenna_svg()}<p class="ad-note">Illustrative geometry &mdash; not computed output</p></div>
    </section>""")

# ----- matplotlib helper
def style_figure(fig):
    """Dark-mode a matplotlib figure AFTER plotting (does not touch data)."""
    fig.patch.set_facecolor(PLOT["bg"])
    for ax in fig.axes:
        ax.set_facecolor(PLOT["axes"])
        ax.grid(True, color="white", alpha=.08, linewidth=.8)
        ax.tick_params(colors=PLOT["text"])
        ax.xaxis.label.set_color(PLOT["text"]); ax.yaxis.label.set_color(PLOT["text"])
        ax.title.set_color(PLOT["title"])
        for sp in ax.spines.values():
            sp.set_color((1, 1, 1, .15))
        leg = ax.get_legend()
        if leg:
            leg.get_frame().set_facecolor(PLOT["bg"]); leg.get_frame().set_edgecolor((1, 1, 1, .15))
            for t in leg.get_texts():
                t.set_color(PLOT["text"])
    return fig

def plot_s11_vswr(freqs_ghz, s11_db, vswr, fr_ghz=None, s11_min_db=None):
    """Side-by-side S11 / VSWR figure in the AntDesigner style. Plots ONLY the arrays you pass in
    (real engine output). Shaded band = where S11 <= -10 dB, shown on both panels for consistency.
    Note: -10 dB ~ VSWR 1.92, so the VSWR=2 line sits marginally outside the band edge."""
    import numpy as np
    import matplotlib.pyplot as plt
    f = np.asarray(freqs_ghz, float); s = np.asarray(s11_db, float); v = np.asarray(vswr, float)
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4.1), constrained_layout=True)
    band = s <= -10
    for ax in (a1, a2):
        ax.fill_between(f, 0, 1, where=band, transform=ax.get_xaxis_transform(), color=PLOT["fill"], lw=0, label="S11 \u2264 \u221210 dB")
        ax.set_xlim(f.min(), f.max()); ax.set_xlabel("Frequency (GHz)")
    a1.plot(f, s, color=PLOT["s11"], lw=2.4, label="S11")
    a1.axhline(-10, color=PLOT["threshold"], ls="--", lw=1.2, label="\u221210 dB")
    a1.set_ylabel("S11 (dB)"); a1.set_title("Return loss (S11)")
    a2.plot(f, v, color=PLOT["vswr"], lw=2.4, label="VSWR")
    a2.axhline(2, color=PLOT["threshold"], ls=":", lw=1.4, label="VSWR = 2")
    a2.set_ylim(1, min(max(5.0, 2.5), max(float(np.nanmax(v)), 3.0))); a2.set_ylabel("VSWR"); a2.set_title("VSWR")
    if fr_ghz is not None:
        i = int(np.nanargmin(np.abs(f - fr_ghz)))

```

```

    a1.scatter([f[i]], [s[i]], s=60, zorder=5, color=PLOT["marker_face"], edgecolor=PLOT["marker_edge"], lw=2)
    lab = f"fr = {fr_ghz:.2f} GHz" + (f", {s11_min_db:.1f} dB" if s11_min_db is not None else "")
    a1.annotate(lab, (f[i], s[i]), xytext=(10, 14), textcoords="offset points", color=PLOT["title"], fontsize=9)
    a2.scatter([f[i]], [min(v[i], a2.get_ylim()[1])], s=60, zorder=5, color=PLOT["marker_face"], edgecolor=PLOT["marker_edge"], lw=2)
    a1.legend(loc="best", fontsize=8); a2.legend(loc="best", fontsize=8)
    return style_figure(fig)

```

```

# ----- entry point

```

```

def apply_theme():
    """CSS + behaviour. Call once, immediately after st.set_page_config()."""
    st.markdown(f"<style>{THEME_CSS}</style>", unsafe_allow_html=True)
    code = json.dumps(PARENT_JS)
    components.html(
        "<script>(function(){try{const d=window.parent.document;if(d.getElementById('ad-js'))return;"
        "const s=d.createElement('script');s.id='ad-js';s.textContent=" + code + ";d.head.appendChild(s);}"
        "catch(e){console.error('AntDesigner theme: cannot reach parent document',e);}})();</script>",
        height=0)

```